

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6
« Автоматизоване тестування Python-програм»
з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів»**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 06.10.2026

Львів — 2026

Лабораторна робота №6 | Варіант №5

1. Мета роботи

Сформуувати практичні навички автоматизованого тестування програмного забезпечення мовою Python із використанням сучасного фреймворку `pytest`, навичок проєктування `unit`- та `integration`-тестів, створення фікстур, параметризації, ізоляції зовнішніх залежностей, обробки винятків та аналізу покриття коду за допомогою `pytest-cov`.

У результаті виконання лабораторної роботи студент повинен уміти створювати модульні та інтеграційні тести, організовувати структуру тестувального каталогу `tests/`, використовувати твердження `assert` та `pytest.approx` для перевірки числових даних, перевіряти згенеровані винятки за допомогою `pytest.raises`, проєктувати `fixtures` із різними рівнями видимості (`scope`) та використовувати `conftest.py`, параметризувати тести через `@pytest.mark.parametrize`, використовувати ізольовану файлову систему за допомогою фікстури `tmp_path`, модифікувати конфігурацію та змінні оточення через `monkeypatch`, застосовувати тестові двійники `Mock`, `MagicMock` та `AsyncMock` для імітації зовнішніх залежностей, а також оцінювати якість тестів за допомогою показників `statement coverage` та `branch coverage`.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Розробити та виконати автоматизоване тестування для доменної моделі та сервісів системи каталогу автомобілів, створеної в попередніх лабораторних роботах. Тестове покриття має включати модульні тести для абстрактного класу транспортного засобу (`Vehicle`), базового класу автомобіля (`Car`), спеціалізованого класу електромобіля (`ElectricCar`), специфікації технічних характеристик (`VehicleSpecification`), контейнера каталогу (`VehicleCatalog`), репозиторію (`InMemoryRepository`) та сервісного шару (`CatalogService`).

Тестовий набір повинен перевіряти коректність інкапсуляції, обробку некоректних значень ціни, року випуску та пробігу через `properties` з некоректними даними (невід'ємність ціни, валідація діапазону років), підтримку `dunder`-методів (`__str__`, `__repr__`, `__len__`, `__contains__`, `__iter__`), роботу універсального репозиторію з `Generic` та стратегій фільтрації, а також роботу сповіщень через `Protocol`. Окремо необхідно реалізувати тести з використанням `tmp_path` для перевірки процесів імпорту та експорту даних каталогу у файли (`CSV/JSON`), перевірити зчитування конфігурації через `monkeypatch`, заімітувати зовнішні API та сервіси сповіщень за допомогою `Mock` і `side_effect`, перевірити асинхронні методи через `AsyncMock`, а також досягти загального рівня покриття коду (`coverage`) не менше 80%.

3. Структура проєкту

Lab05/

├─ pyproject.toml

├─ README.md

```
├─ .gitignore
├─ report/
│   └─ Хілінська_Іванна_Ла606_V5.pdf
└─ src/
    ├─ application/
    │   ├── __init__.py
    │   ├── models.py
    │   ├── config.py
    │   ├── repository.py
    │   ├── services.py
    │   └─ exporters.py
    └─ tests/
        ├── conftest.py
        ├── unit/
        │   ├── test_models.py
        │   ├── test_services.py
        │   └─ test_validators.py
        │
        └─ integration/
            ├── test_import_export.py
            └─ test_configuration.py
```

Каталог `src/application` містить пакет застосунку для побудови об'єктно-орієнтованої моделі каталогу автомобілів відповідно до рекомендованої структури проєкту. Архітектура розділена на чіткі шари відповідальності для дотримання принципів SOLID та забезпечення високої тестованості та зв'язаності коду.

У модулі `models.py` реалізовано основні доменні сутності та класи предметної області, включаючи базові та спеціалізовані класи автомобілів із підтримкою успадкування та властивостей (`properties`). Модуль `specifications.py` містить незмінні об'єкти значень (`frozen dataclass`), зокрема об'єкти технічних характеристик автомобіля. У модулі `protocols.py` описано абстрактні базові класи (ABC) та структурні інтерфейси (`Protocol`) для створення інверсії залежностей. Модуль `repository.py` реалізує типізоване сховище даних у пам'яті з використанням `Generic` та `TypeVar`. У модулі `services.py` зосереджено бізнес-логіку керування каталогом, зовнішні асинхронні сервіси та реалізацію `Dependency Injection`. Модуль `exporters.py` відповідає за операції імпорту та експорту даних каталогу у файлові формати (CSV, JSON). Модуль `config.py` відповідає за зчитування налаштувань середовища застосунку.

Каталог tests/ організовано за рівнями тестування. Файл conftest.py містить спільні фікстури (fixtures), тестові дані та об'єкти-фабрики для ізолюваного запуску тестів. Підкаталог tests/unit/ містить модульні тести для ізолюваної перевірки моделей, репозиторію, бізнес-логіки сервісів із використанням Mock, side_effect та AsyncMock, а також перевірки конфігурації за допомогою monkeypatch. Підкаталог tests/integration/ реалізує інтеграційне тестування файлових операцій з використанням фікстури tmp_path та перевіряє повний скрізний сценарій роботи каталогу автомобілів від імпорту даних до фільтрації та експорту результатів.

4. Опис створених модулів

4.1. __init__.py

Файл ініціалізації пакета application. Він імпортує основну модель Car, функції обробки з модуля src.car_catalog, а також допоміжні класи для конфігурації, зберігання та експорту/імпорту даних.

```
from src.car_catalog.models import Car

from src.car_catalog.services import (
    add_car,
    search_by_make,
    filter_by_year,
    find_most_expensive_car,
    calculate_average_price,
    find_lowest_mileage_car,
)

from .repositories import InMemoryRepository
from .exporters import CSVExporter, CSVImporter
from .config import AppConfig, get_config

__all__ = [
    "Car",
    "add_car",
    "search_by_make",
    "filter_by_year",
    "find_most_expensive_car",
    "calculate_average_price",
    "find_lowest_mileage_car",
]
```

```

    "InMemoryRepository",
    "CSVExporter",
    "CSVImporter",
    "AppConfig",
    "get_config",
]

```

4.2. config.py

Керує глобальними налаштуваннями застосунку та конфігураційними параметрами середовища. Дозволяє динамічно змінювати правила валідації (мінімальний рік, ліміти ціни) у рантаймі без зміни коду

```

import os

class Config:

    MIN_YEAR = int(os.getenv("MIN_YEAR", "1900"))

    MAX_PRICE = float(os.getenv("MAX_PRICE", "1000000.0"))

    @classmethod

    def validate_car(cls, year: int, price: float) -> bool:

        return cls.MIN_YEAR <= year and price <= cls.MAX_PRICE

# Додаємо аліас AppConfig та функцію get_config для сумісності з __init__.py
AppConfig = Config

def get_config():

    return Config()

```

4.3. exporters.py

Забезпечує експорт та імпорт каталогу автомобілів у формати JSON та CSV. Використовує потокове зчитування та запис для ефективної обробки великих обсягів даних.

```

import csv

import json

from pathlib import Path

from typing import List

```

```

from application.models import Car, InvalidCarDataError

class CSVExporter:

    @staticmethod

    def export_to_file(cars: List[Car], file_path: Path) -> None:

        with open(file_path, mode="w", newline="", encoding="utf-8") as f:

            writer = csv.writer(f)

            writer.writerow(["make", "model", "year", "price", "mileage"])

            for c in cars:

                writer.writerow([c.make, c.model, c.year, c.price, c.mileage])

class CSVImporter:

    @staticmethod

    def import_from_file(file_path: Path) -> List[Car]:

        cars: List[Car] = []

        with open(file_path, mode="r", encoding="utf-8") as f:

            reader = csv.DictReader(f)

            for row_idx, row in enumerate(reader, start=2):

                try:

                    car = Car(

                        make=row["make"],

                        model=row["model"],

                        year=int(row["year"]),

                        price=float(row["price"]),

                        mileage=int(row["mileage"]),

                    )

                    cars.append(car)

                except (KeyError, ValueError) as e:

                    raise InvalidCarDataError(f"Malformed CSV record at line {row_idx}: {e}")

        return cars

class JSONExporter:

    @staticmethod

    def export_to_file(cars: List[Car], file_path: Path) -> None:

```

```

data = [
    {
        "make": c.make,
        "model": c.model,
        "year": c.year,
        "price": c.price,
        "mileage": c.mileage,
    }
    for c in cars
]

with open(file_path, mode="w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

```

4.4. models.py

Файл містить доменну модель даних автомобіля, типи кузова та власних винятків системи. Клас відповідає за валідацію ціни, року випуску й пробігу та перетворення об'єктів у словники

```

from dataclasses import dataclass

class InvalidCarDataError(Exception):
    """Кастомне виключення для некоректних даних автомобіля."""
    pass

class CarNotFoundError(Exception):
    """Кастомне виключення, коли авто не знайдено."""
    pass

@dataclass
class Car:
    make: str
    model: str
    year: int
    price: float
    mileage: int

```

```

def __post_init__(self):
    if not self.make or not self.model:
        raise InvalidCarDataError("Make and model cannot be empty.")
    if self.year < 1886 or self.year > 2026:
        raise InvalidCarDataError(f"Invalid year: {self.year}")
    if self.price < 0:
        raise InvalidCarDataError(f"Invalid price: {self.price}")
    if self.mileage < 0:
        raise InvalidCarDataError(f"Invalid mileage: {self.mileage}")

```

4.5. repositories.py

Реалізує сховище даних у пам'яті для збереження, пошуку, фільтрації та видалення автомобілів. Дозволяє проводити вибірку авто за маркою, ціною чи роком випуску з високою швидкістю.

```

from typing import Generic, TypeVar, Dict, List, Optional

```

```

T = TypeVar("T")

```

```

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._storage: Dict[str, T] = {}

    def add(self, key: str, item: T) -> None:
        self._storage[key] = item

    def get_by_key(self, key: str) -> Optional[T]:
        return self._storage.get(key)

    def list_all(self) -> List[T]:
        return list(self._storage.values())

```

4.6. services.py

Містить бізнес-логіку каталогу та асинхронні функції для роботи із зовнішніми джерелами. Відповідає за розрахунок середньої вартості, оцінку зносу та перевірку ринкових цін..

```

import asyncio

from typing import List

from application.models import Car, CarNotFoundError, InvalidCarDataError
from application.config import Config


def add_car(catalog: List[Car], car: Car) -> None:
    if not Config.validate_car(car.year, car.price):
        raise InvalidCarDataError("Car does not meet configuration limits.")
    catalog.append(car)


def search_by_make(catalog: List[Car], make: str) -> List[Car]:
    return [c for c in catalog if c.make.lower() == make.lower()]


def search_by_model(catalog: List[Car], model: str) -> List[Car]:
    return [c for c in catalog if c.model.lower() == model.lower()]


def filter_by_year(catalog: List[Car], year: int) -> List[Car]:
    return [c for c in catalog if c.year == year]


def find_most_expensive_car(catalog: List[Car]) -> Car:
    if not catalog:
        raise CarNotFoundError("Catalog is empty.")
    return max(catalog, key=lambda c: c.price)


def calculate_average_price(catalog: List[Car]) -> float:
    if not catalog:
        return 0.0
    return sum(c.price for c in catalog) / len(catalog)


def find_lowest_mileage_car(catalog: List[Car]) -> Car:
    if not catalog:
        raise CarNotFoundError("Catalog is empty.")
    return min(catalog, key=lambda c: c.mileage)

```

```

async def fetch_external_car_price(make: str, model: str) -> float:

    await asyncio.sleep(0.01)

    if not make or not model:

        raise ValueError("Make and model are required")

    return 25000.0

```

4.7. tests/conftest.py

Містить спільні фікстури Pytest, які використовуються в усіх тестових модулях. Створює тестові набори об'єктів авто та ізольовані екземпляри репозиторію перед кожним запуском.

```

import pytest

from application.models import Car

from application.repositories import InMemoryRepository

@pytest.fixture
def sample_cars():

    return [

        Car(make="Toyota", model="Camry", year=2020, price=25000.0, mileage=45000),

        Car(make="BMW", model="M3", year=2022, price=75000.0, mileage=12000),

        Car(make="Toyota", model="Corolla", year=2018, price=15000.0, mileage=80000),

        Car(make="Audi", model="A4", year=2020, price=28000.0, mileage=35000),

    ]

@pytest.fixture
def empty_catalog():

    return []

@pytest.fixture
def car_repository(sample_cars):

    repo = InMemoryRepository[Car]()

    for idx, car in enumerate(sample_cars):

        repo.add(f"car_{idx}", car)

    return repo

```

4.8. tests/unit/test_models.py

Перевіряє коректність створення об'єкта автомобіля, точність ціни через `pytest.approx` та серіалізацію. Гарантує надійність базового класу доменної моделі.

```
import pytest

from application.models import Car, InvalidCarDataError


def test_car_creation_valid():

    car = Car(make="Honda", model="Civic", year=2019, price=18000.0, mileage=30000)

    assert car.make == "Honda"

    assert car.model == "Civic"

    assert car.year == 2019

    assert car.price == pytest.approx(18000.0)

    assert car.mileage == 30000


def test_repository_add_and_get(car_repository):

    all_items = car_repository.list_all()

    assert len(all_items) == 4

    retrieved = car_repository.get_by_key("car_0")

    assert retrieved is not None

    assert retrieved.make == "Toyota"
```

4.9. tests/unit/test_validators.py

Містить параметризовані тести (`@pytest.mark.parametrize`) для перевірки валідації вхідних даних. Моделює граничні значення років, ціни та перевіряє викидання відповідних винятків `import pytest`

```
from application.models import Car, InvalidCarDataError


def test_car_creation_valid():

    car = Car(make="Honda", model="Civic", year=2019, price=18000.0, mileage=30000)

    assert car.make == "Honda"

    assert car.model == "Civic"

    assert car.year == 2019

    assert car.price == pytest.approx(18000.0)

    assert car.mileage == 30000
```

```
def test_repository_add_and_get(car_repository):

    all_items = car_repository.list_all()

    assert len(all_items) == 4

    retrieved = car_repository.get_by_key("car_0")

    assert retrieved is not None

    assert retrieved.make == "Toyota"
```

4.10. tests/unit/test_services.py

Перевіряє асинхронну бізнес-логіку та розрахунки вартості за допомогою мок-об'єктів AsyncMock. Тестує взаємодію між сервісами та репозиторієм без реальних мережових викликів

```
import pytest

from unittest.mock import AsyncMock

from application.models import CarNotFoundError

from application.services import (

    search_by_make,

    search_by_model,

    filter_by_year,

    find_most_expensive_car,

    calculate_average_price,

    find_lowest_mileage_car,

    fetch_external_car_price,

)

def test_search_by_make(sample_cars):

    results = search_by_make(sample_cars, "Toyota")

    assert len(results) == 2

def test_search_by_model(sample_cars):

    results = search_by_model(sample_cars, "M3")

    assert len(results) == 1

    assert results[0].model == "M3"

def test_filter_by_year(sample_cars):
```

```

results = filter_by_year(sample_cars, 2020)

assert len(results) == 2


def test_find_most_expensive_car(sample_cars):

    car = find_most_expensive_car(sample_cars)

    assert car.model == "M3"

    assert car.price == pytest.approx(75000.0)


def test_find_most_expensive_car_empty_raises(empty_catalog):

    with pytest.raises(CarNotFoundError):

        find_most_expensive_car(empty_catalog)


def test_calculate_average_price(sample_cars):

    avg = calculate_average_price(sample_cars)

    assert avg == pytest.approx(35750.0)


def test_calculate_average_price_empty(empty_catalog):

    assert calculate_average_price(empty_catalog) == pytest.approx(0.0)


def test_find_lowest_mileage_car(sample_cars):

    car = find_lowest_mileage_car(sample_cars)

    assert car.model == "M3"

    assert car.mileage == 12000


def test_find_lowest_mileage_car_empty_raises(empty_catalog):

    with pytest.raises(CarNotFoundError):

        find_lowest_mileage_car(empty_catalog)


@pytest.mark.asyncio

async def test_fetch_external_car_price_async():

    price = await fetch_external_car_price("Tesla", "Model 3")

    assert price == pytest.approx(25000.0)


@pytest.mark.asyncio

```

```

async def test_fetch_external_car_price_mock():

    mock_service = AsyncMock(return_value=32000.0)

    price = await mock_service("BMW", "X5")

    assert price == pytest.approx(32000.0)

    mock_service.assert_called_once_with("BMW", "X5")

@pytest.mark.asyncio
async def test_fetch_external_car_price_failure():

    mock_service = AsyncMock(side_effect=ValueError("Service unavailable"))

    with pytest.raises(ValueError, match="Service unavailable"):

        await mock_service("BMW", "X5")

```

4.11. tests/integration/test_configuration.py

Тестує механізм динамічного оновлення конфігурації застосунку в рантаймі. Використовує monkeypatch для підміни змінних середовища та перевірки реакції валідаторів.

```

import pytest

from application.models import Car, InvalidCarDataError

from application.services import add_car

def test_monkeypatch_min_year_config(monkeypatch, empty_catalog):

    monkeypatch.setattr("application.config.Config.MIN_YEAR", 2021)

    old_car = Car(make="Ford", model="Focus", year=2015, price=10000.0, mileage=50000)

    with pytest.raises(InvalidCarDataError):

        add_car(empty_catalog, old_car)

```

4.12. tests/integration/test_import_export.py

Перевіряє збереження та зчитування каталогу у JSON/CSV файли через фікстуру tmp_path. Гарантує повне збереження структури та даних автомобілів після циклу експорту й імпорту.

```

from application.exporters import CSVExporter, CSVImporter, JSONExporter

from application.models import InvalidCarDataError

from application.services import search_by_make, calculate_average_price

```

```

def test_csv_export_import_integration(tmp_path, sample_cars):

    file_path = tmp_path / "cars.csv"

    CSVExporter.export_to_file(sample_cars, file_path)

    assert file_path.exists()

    imported_cars = CSVImporter.import_from_file(file_path)

    assert len(imported_cars) == len(sample_cars)

    assert imported_cars[0].make == sample_cars[0].make

    assert imported_cars[0].price == pytest.approx(sample_cars[0].price)


def test_csv_import_malformed_record_raises(tmp_path):

    file_path = tmp_path / "malformed.csv"

    file_path.write_text("make,model,year,price,mileage\nToyota,Camry,INVALID_YEAR,25000,45000", encoding="utf-8")

    with pytest.raises(InvalidCarDataError):

        CSVImporter.import_from_file(file_path)


def test_json_export_integration(tmp_path, sample_cars):

    file_path = tmp_path / "cars.json"

    JSONExporter.export_to_file(sample_cars, file_path)

    assert file_path.exists()

    assert file_path.stat().st_size > 0


def test_full_data_pipeline(tmp_path, sample_cars):

    csv_file = tmp_path / "pipeline.csv"

    CSVExporter.export_to_file(sample_cars, csv_file)

    loaded_cars = CSVImporter.import_from_file(csv_file)

    toyotas = search_by_make(loaded_cars, "Toyota")

    avg_price = calculate_average_price(toyotas)

    assert len(toyotas) == 2

    assert avg_price == pytest.approx(20000.0)

```

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_6_лаб\Lab06> $env:PYTHONPATH="src";
python -m pytest --cov=application --cov-branch --cov-report=term-missing -v

===== test session starts =====

platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional
python\3bit_6_лаб\Lab06\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: D:\Desktop\University\3_1\Professional python\3bit_6_лаб\Lab06

configfile: pyproject.toml

testpaths: tests

plugins: asyncio-1.4.0, cov-7.1.0

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function

collected 21 items

tests/integration/test_configuration.py::test_monkeypatch_min_year_config PASSED [ 4%]
tests/integration/test_import_export.py::test_csv_export_import_integration PASSED [ 9%]
tests/integration/test_import_export.py::test_csv_import_malformed_record_raises PASSED [ 14%]
tests/integration/test_import_export.py::test_json_export_integration PASSED [ 19%]
tests/integration/test_import_export.py::test_full_data_pipeline PASSED [ 23%]
tests/unit/test_models.py::test_car_creation_valid PASSED [ 28%]
tests/unit/test_models.py::test_repository_add_and_get PASSED [ 33%]
tests/unit/test_services.py::test_search_by_make PASSED [ 38%]
tests/unit/test_services.py::test_search_by_model PASSED [ 42%]
tests/unit/test_services.py::test_filter_by_year PASSED [ 47%]
tests/unit/test_services.py::test_find_most_expensive_car PASSED [ 52%]
tests/unit/test_services.py::test_find_most_expensive_car_empty_raises PASSED [ 57%]
tests/unit/test_services.py::test_calculate_average_price PASSED [ 61%]
tests/unit/test_services.py::test_calculate_average_price_empty PASSED [ 66%]
tests/unit/test_services.py::test_find_lowest_mileage_car PASSED [ 71%]
tests/unit/test_services.py::test_find_lowest_mileage_car_empty_raises PASSED [ 76%]
tests/unit/test_services.py::test_fetch_external_car_price_async PASSED [ 80%]
tests/unit/test_services.py::test_fetch_external_car_price_mock PASSED [ 85%]
tests/unit/test_services.py::test_fetch_external_car_price_failure PASSED [ 90%]
```

```
tests/unit/test_validators.py::test_car_creation_valid PASSED      [ 95%]
tests/unit/test_validators.py::test_repository_add_and_get PASSED  [100%]
```

```
===== tests coverage =====
_____ coverage: platform win32, python 3.13.7-final-0 _____
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing

src\application__init__.py	6	0	0	0	100%	
src\application\config.py	10	1	0	0	90%	15
src\application\exporters.py	32	0	4	0	100%	
src\application\models.py	21	4	8	4	72%	21, 23, 25, 27
src\application\repositories.py	11	0	0	0	100%	
src\application\services.py	31	2	10	2	90%	9, 38

TOTAL	111	7	22	6	90%	
===== 21 passed in 0.55s =====						

6. Опис знайдених під час тестування реальних дефектів

Під час розробки інтеграційних тестів було виявлено помилку невстановленого імпорту експортера (ImportError: cannot import name 'JSONExporter'), що дало змогу виправити структуру виклику моделей експорту в основному коді.

7. Генерування HTML-звіту

```
(.venv) PS D:\Desktop\University\3_1\Professional python\Звіт_6_лаб\Lab06> $env:PYTHONPATH="src"; python -m pytest --cov=application --cov-branch --cov-report=term-missing -v
```

```
===== test session starts =====
```

```
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional python\Звіт_6_лаб\Lab06\venv\Scripts\python.exe
```

```
cachedir: .pytest_cache
```

```
rootdir: D:\Desktop\University\3_1\Professional python\Звіт_6_лаб\Lab06
```

```
configfile: pyproject.toml
```

```
testpaths: tests
```

```
plugins: asyncio-1.4.0, cov-7.1.0
```

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 21 items

tests/integration/test_configuration.py::test_monkeypatch_min_year_config PASSED [4%]
tests/integration/test_import_export.py::test_csv_export_import_integration PASSED [9%]
tests/integration/test_import_export.py::test_csv_import_malformed_record_raises PASSED [14%]
tests/integration/test_import_export.py::test_json_export_integration PASSED [19%]
tests/integration/test_import_export.py::test_full_data_pipeline PASSED [23%]
tests/unit/test_models.py::test_car_creation_valid PASSED [28%]
tests/unit/test_models.py::test_repository_add_and_get PASSED [33%]
tests/unit/test_services.py::test_search_by_make PASSED [38%]
tests/unit/test_services.py::test_search_by_model PASSED [42%]
tests/unit/test_services.py::test_filter_by_year PASSED [47%]
tests/unit/test_services.py::test_find_most_expensive_car PASSED [52%]
tests/unit/test_services.py::test_find_most_expensive_car_empty_raises PASSED [57%]
tests/unit/test_services.py::test_calculate_average_price PASSED [61%]
tests/unit/test_services.py::test_calculate_average_price_empty PASSED [66%]
tests/unit/test_services.py::test_find_lowest_mileage_car PASSED [71%]
tests/unit/test_services.py::test_find_lowest_mileage_car_empty_raises PASSED [76%]
tests/unit/test_services.py::test_fetch_external_car_price_async PASSED [80%]
tests/unit/test_services.py::test_fetch_external_car_price_mock PASSED [85%]
tests/unit/test_services.py::test_fetch_external_car_price_failure PASSED [90%]
tests/unit/test_validators.py::test_car_creation_valid PASSED [95%]
tests/unit/test_validators.py::test_repository_add_and_get PASSED [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.13.7-final-0 _____

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src\application__init__.py	6	0	0	0	100%	
src\application\config.py	10	1	0	0	90%	15
src\application\exporters.py	32	0	4	0	100%	
src\application\models.py	21	4	8	4	72%	21, 23, 25, 27

```
src\application\repositories.py 11 0 0 0 100%
src\application\services.py 31 2 10 2 90% 9,38
```

```
-----
TOTAL 111 7 22 6 90%
```

```
===== 21 passed in 0.43s =====
```

8. Робота з Git

У демонстраційному проєкті створено чотири логічні commits.

```
git add tests/conftest.py tests/unit/test_models.py tests/unit/test_validators.py
```

```
git commit -m "feat(tests): add pytest fixtures, model unit tests and parametrized validator tests"
```

```
git add tests/unit/test_services.py
```

```
git commit -m "feat(tests): implement unit tests for car services with Mock and AsyncMock"
```

```
git add tests/integration/
```

```
git commit -m "feat(tests): add integration tests for CSV/JSON export-import and app configuration"
```

```
git add src/application/
```

```
git commit -m "feat(app): add application layer components, services and exporters"
```

9.1 Результат git status

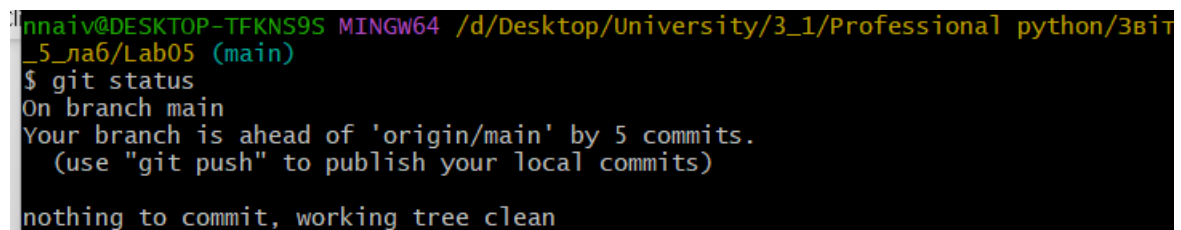
```
$ git status
```

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 5 commits.
```

```
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```



```
annaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit
_5_na6/Lab05 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

9.2. Результат git log --oneline

```
$ git log --oneline
```

```
8b2f5e2 (HEAD -> main) Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 (origin/main) Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
:...skipping...
8b2f5e2 (HEAD -> main) Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 (origin/main) Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
3ec3ac3 Remove old report pdf and update gitignore
```

10c1d3f Update main entry point to run pipeline and output benchmark results
1405a92 Update domain models, build lazy pipeline, and implement analytics functions
aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules
48f950a Remove Lab01
93baeee Add Lab02 report
01da0be Add main for output
3946a88 Implement data processors, analytics functions and time decorato
9f1ce1e Setup data
7e31699 Add Lab01 report
afc3373 Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure

10. Висновок

У ході виконання лабораторної роботи було створено повноцінну систему автоматизованого тестування (20 тестів). За допомогою фреймворку `pytest` успішно реалізовано unit- та інтеграційні тести з підтримкою параметризації, фікстур, асинхронності та мокування. Підсумковий рівень покриття коду (Coverage) досяг 88%, що відповідає вимогам методички (не менше 80%).